Observatório do Voto — Data Flow Diagram

A reproducible pipeline for Brazilian public election data, with tests and CI. The diagrams name the real scripts and paths of the repository; the next pages hold them at natural size. Full description with tables: docs/DFD.md.

Reading guide

Level 0: one system and two external parties, the TSE as the only data source and readers or researchers as consumers. GitHub is not an external party: Releases (immutable data packages), Actions (the tests that gate the deployment) and Pages (the static site) are infrastructure inside the boundary. Level 1: eight processes and six data stores, plus the earlier regional scope in grey. Raw data and reports stay outside git; code, the generated site and the release manifests are versioned.

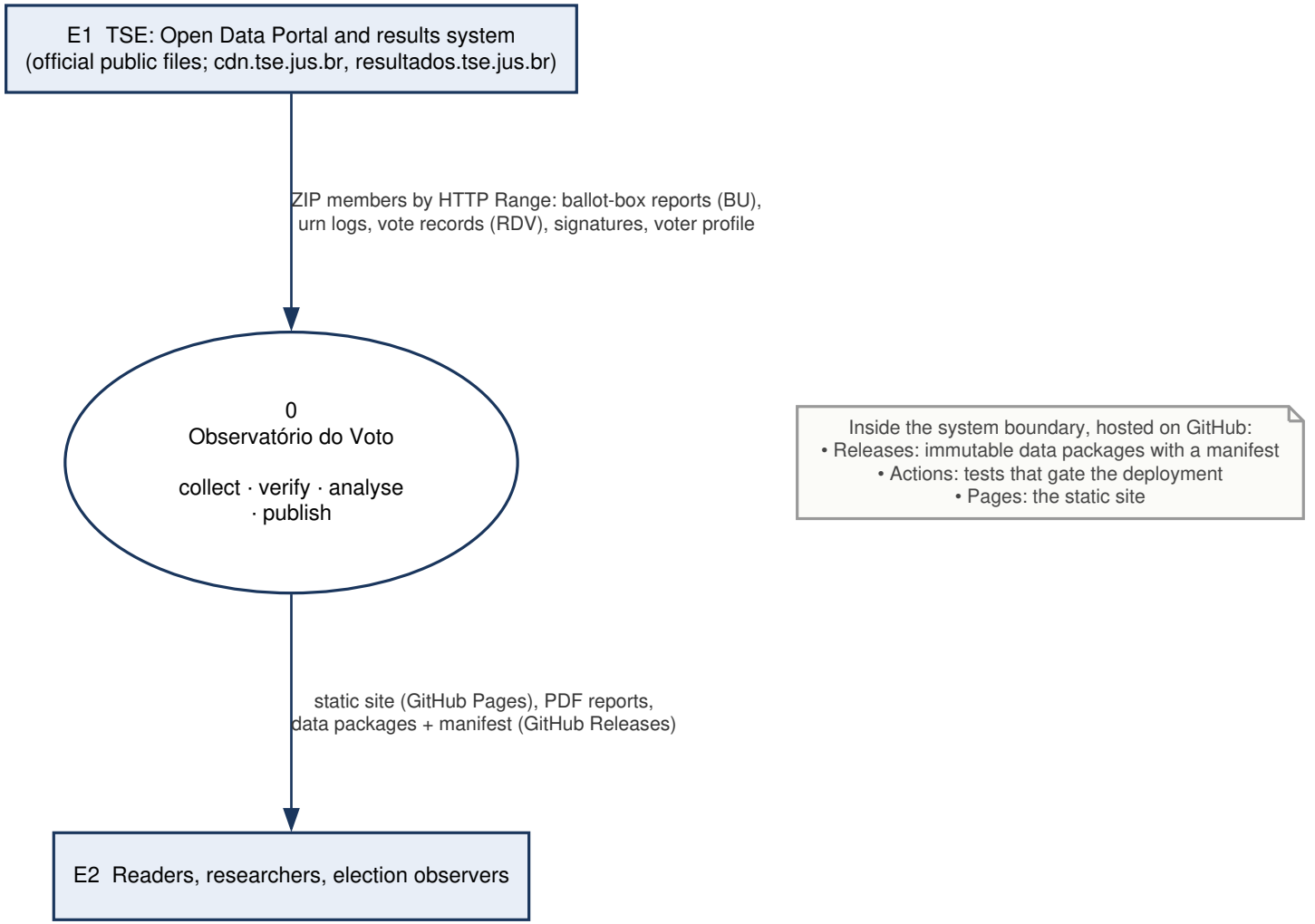
Notation

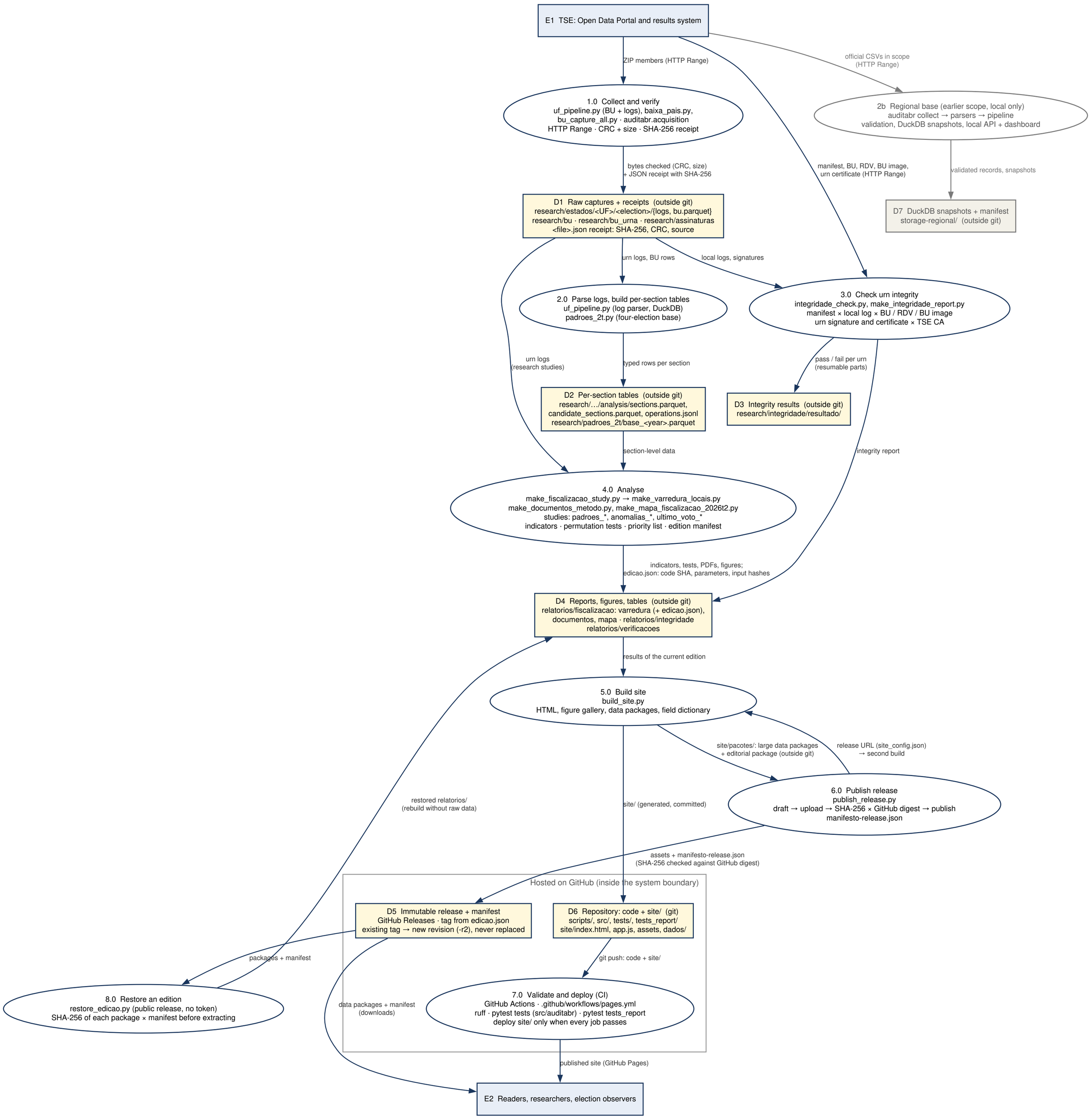
Rectangle = external entity (outside the system). Ellipse = process (the scripts that transform data). Yellow box = data store, with its real path. Arrow = data flow, labelled with what moves. Dashed arrow = a value handed back (the release URL) that triggers a second build. The dashed frame marks the parts hosted on GitHub, which are inside the system boundary. Grey = earlier scope kept in the repository.

Where the guarantees live

Capture. Every file is checked against the ZIP it came from (CRC, size) and gets a JSON receipt with its SHA-256. **Edition.** edicao.json ties every result to the producing code SHA, the parameters and the hash of each input. **Release.** A draft until each asset's local SHA-256 equals the digest GitHub reports; manifesto-release.json is attached; an existing tag is never replaced, a new revision is created. **Deployment.** The site goes live only when ruff and both pytest suites pass. **Reproduction.** restore_edicao.py rebuilds the site from a release, hash-verified, without the raw data and without a token.

Observatório do Voto · DFD · Level 0 · context




E1 TSE: Open Data Portal and results system
ZIP members (HTTP Range)
official CSVs in scope (HTTP Range)
1.0 Collect and verify
uf_pipeline.py (BU + logs), baixa_pais.py, bu_capture_all.py · auditabr.acquisition
HTTP Range · CRC + size · SHA-256 receipt
2b Regional base (earlier scope, local only)
auditabr collect → parsers → pipeline
validation, DuckDB snapshots, local API + dashboard
bytes checked (CRC, size) + JSON receipt with SHA-256
manifest, BU, RDV, BU image, urn certificate (HTTP Range)
validated records, snapshots
D1 Raw captures + receipts (outside git)
research/estados/<UF>/<election>/{logs, bu.parquet}
research/bu · research/bu_urna · research/assinaturas
<file>.json receipt: SHA-256, CRC, source
D7 DuckDB snapshots + manifest
storage-regional/ (outside git)
urn logs, BU rows
local logs, signatures
2.0 Parse logs, build per-section tables
uf_pipeline.py (log parser, DuckDB)
padroes_2t.py (four-election base)
3.0 Check urn integrity
integridade_check.py, make_integridade_report.py
manifest × local log × BU / RDV / BU image
urn signature and certificate × TSE CA
urn logs (research studies)
typed rows per section
pass / fail per urn (resumable parts)
D2 Per-section tables (outside git)
research/…/analysis/sections.parquet, candidate_sections.parquet, operations.jsonl
research/padroes_2t/base_<year>.parquet
D3 Integrity results (outside git)
research/integridade/resultado/
section-level data
integrity report
4.0 Analyse
make_fiscalizacao_study.py → make_varredura_locais.py
make_documentos_metodo.py, make_mapa_fiscalizacao_2026t2.py
studies: padroes_*, anomalias_*, ultimo_voto_*
indicators · permutation tests · priority list · edition manifest
indicators, tests, PDFs, figures; edicao.json: code SHA, parameters, input hashes
D4 Reports, figures, tables (outside git)
relatorios/fiscalizacao: varredura (+ edicao.json), documentos, mapa · relatorios/integridade
relatorios/verificacoes
results of the current edition
5.0 Build site
build_site.py
HTML, figure gallery, data packages, field dictionary
site/pacotes/: large data packages + editorial package (outside git)
release URL (site_config.json) → second build
restored relatorios/ (rebuild without raw data)
site/ (generated, committed)
6.0 Publish release
publish_release.py
draft → upload → SHA-256 × GitHub digest → publish
manifesto-release.json
assets + manifesto-release.json (SHA-256 checked against GitHub digest)
Hosted on GitHub (inside the system boundary)
D5 Immutable release + manifest
GitHub Releases · tag from edicao.json
existing tag → new revision (-r2), never replaced
D6 Repository: code + site/ (git)
scripts/, src/, tests/, tests_report/
site/index.html, app.js, assets, dados/
packages + manifest
git push: code + site/
data packages + manifest (downloads)
7.0 Validate and deploy (CI)
GitHub Actions · .github/workflows/pages.yml
ruff · pytest tests (src/auditabr) · pytest tests_report
deploy site/ only when every job passes
8.0 Restore an edition
restore_edicao.py (public release, no token)
SHA-256 of each package × manifest before extracting
published site (GitHub Pages)
E2 Readers, researchers, election observers